Who Wrote It? Identifying LLMs from Their Responses

A Project Authored By

Nicholas Gauthier

9 October 2026

Problem Definition and Dataset Curation

The purpose of this project is to answer the question can machine learning identify the AI behind the text? To answer this question, we must create two models, a Convolutional Neural Network (CNN) and a Recurrent Neural Network (RNN) and evaluate how accurate they are at predicting which LLM wrote the response based on the response, then based on the question, and then finally based on both the question and the response. The first problem is that we must find the data to train these models on and the data I found comes from this link, TXD-22: A Large-Scale Benchmark Dataset for Multi-Class AI-Generated and Mixed-Source Text Identification - Mendeley Data. This dataset has 66000 rows where it asks 22 different LLMs 3000 questions each. For my project I decided to focus on 4 LLMs, ChatGPT, Grok, Copilot, and DeepSeek so I filtered this dataset for just the responses from those LLMs and divided it so that it only took questions that appeared in all 4 datasets. Now we have a dataset with 11840 rows where 2960 questions are all repeated 4 times, so we see the response each of the 4 LLMs gave to the same question. Now to create the training, validation, and testing csv files, they were split so that the same question would always appear in the same dataset to prevent the model from learning from the input. This gave us a training csv where 9472 rows had 2368 questions repeated 4 times, a validation csv where 1184 rows had 296 questions repeated 4 times, and a testing csv where 1184 rows had 296 questions repeated 4 times. The next step would be creating the dataset and because our question asks us to compare the models using just the output, just the input, and both the input and output we must create three different datasets, one for each of those scenarios. After cleaning some small issues like whitespace and separating each response into a list of the words in that response the list of words was tokenized into a numerical ID with a maximum vocabulary size of 30,000. The response that would be tokenized depended on which code we looked at as the output dataset just tokenizes the response, the input dataset just tokenizes the question, and the dataset with both tokenizes both the question then adds a separator then also tokenizes the response. It does this for the training, validation, and testing csv files so now we have a tokenized dataset for each stage of model building for response, question, and both question and response.

CNN/RNN Implementation and Training

Now that the datasets have been created we must create and train the models. I created a CNN model and an RNN model, which was implemented using a Long Short-Term Memory (LSTM) network. My CNN model used kernel sizes of 3, 4, and 5. This means that it would look at the surrounding 3, 4, or 5 tokens to try and recognize patterns based on how often tokens show up together and from that it would classify a model which it thinks is most likely to use said tokens in said order. My RNN model used a bidirectional LSTM with a hidden dimension of 128. While my CNN model uses the kernel size to detect local patterns, the LSTM is bidirectional, meaning it processes the text both forward and backwards, maintaining information about all the numerical IDs that are in one response. Both models use a dropout rate of 0.5 to help reduce overfitting, they were also both trained for 10 epochs using a learning rate of 0.001. After each epoch, the validation dataset was used to determine the best current model, which would then be saved. After all, 10 epochs the best model found using the validation dataset would be tested on the testing dataset so we could evaluate how well the model could classify data it had not trained on. The code for my CNN and LSTM models imports from the dataset python files so to create models for just the response, just the question, and both the question and response all that we had to do was change which dataset the CNN and LSTM code was importing from.

Results and Analysis

After running the CNN and LSTM code using each of the three datasets, we must now evaluate these models. The clearest way to evaluate the CNN and RNN models is by how often they classify the correct LLM, which is shown in this first table.

Model	**Question only**	**Answer only**	**Question + Answer**
CNN	25.00%	78.80%	79.48%
RNN	25.00%	73.82%	70.35%

This first table shows that neither model was able to predict anything using just the question as both picked correctly 25% which is the same as guessing as there are 4 LLMs, giving a guess a 1/4 chance of being correct. Using just the answer, the CNN model was slightly more accurate, but both models performed very well correctly classifying over 70% of the time. When giving the models both the question and the answer the CNN model became slightly better while the RNN model became slightly worse but overall, both models perform quite well classifying over 70% of the time with CNN almost reaching 80%. These percentages show that the models can both learn to predict which model a response came from and based off the fact that the CNN model is slightly more accurate than the RNN model the slightly bigger tell in which model is being used is which words it puts closer together rather than the entire sentence structure. To answer our two research questions based on these accuracy percentages we can use a CNN and/or an RNN model to classify which LLM generated a response over 70% of the time as this project has shown that we can accurately predict which of Grok, ChatGPT, Copilot, or DeepSeek wrote the output based on that output. The models could not classify which LLM generated a response using only the input as the input was the same for all 4 models so, for example, the model would see a question from the testing csv 4 times, each question would come from a different model but it would guess the same model for all 4 guesses as it has the same input so it's accuracy percentage was 25% for both models, the same as just guessing. The impact of seeing both the question and the answer was limited on both models and I would predict that with more tests the impact it has on the CNN model would likely be about 0. I thought that it may be more useful for the RNN model, but it ultimately was even more damaging for the RNN model as the question is just useless information as it can't predict anything from it as the same question appears for all 4 models. The token limit of 512 and the LSTM dimension of 128 could also play a factor if any response is very long then it would be cut off just so we could include this question information, which we have already established is useless.

Now to dig deeper into that accuracy we can use Confusion Matrices to see where the model was predicting correctly along with where it was being tripped up the most. These confusion matrices are set up so that the guesses are the columns and the

actual answer is the row so if the answer is right it will be in the same LLM for both row and column but if it guessed wrong we can see what it guessed based on which column it is in.

CNN Output Only Confusion Matrix

	ChatGPT	Grok	Copilot	DeepSeek
ChatGPT	185	1	90	20
Grok	3	254	16	23
Copilot	52	2	227	15
DeepSeek	9	4	16	267

RNN Output Only Confusion Matrix

	ChatGPT	Grok	Copilot	DeepSeek
ChatGPT	185	6	83	22
Grok	1	272	2	21
Copilot	97	4	174	21
DeepSeek	13	28	12	243

Starting with just the response confusion matrices for both models we can see that both models were the most successful at classifying Grok and DeepSeek with CNN being better at DeepSeek and RNN being better at Grok. Both models struggled a little bit more with ChatGPT and Copilot and we can see that is because these two are very similar in writing style, meaning they are more likely to pick the same set of words and use them in a more similar order compared to Grok and DeepSeek which are more unique, using more unique words and combinations of words. This meant the models would often pick ChatGPT when it was Copilot and vice versa. The models that are the most different are Grok and ChatGPT as the models would both rarely guess incorrectly that it was Grok when it was ChatGPT or vice versa.

CNN Input Only Confusion Matrix

	ChatGPT	Grok	Copilot	DeepSeek
ChatGPT	45	174	61	16
Grok	45	174	61	16
Copilot	45	174	61	16
DeepSeek	45	174	61	16

RNN Input Only Confusion Matrix

	ChatGPT	Grok	Copilot	DeepSeek
ChatGPT	85	12	1	198
Grok	85	12	1	198
Copilot	85	12	1	198
DeepSeek	85	12	1	198

The input only confusion matrices for both models show exactly what I said earlier about how the model would see each question 4 times, but each question would come from a different model, but it would guess the same model for all 4 guesses. Here we can see like how for 174 of the questions the CNN model correctly thought it was Grok, but because it saw each of those 174 questions 4 times, we can see it correctly guessed Grok 174 times but it also incorrectly guessed Grok when it was ChatGPT, Copilot, or DeepSeek 174 times each.

CNN Input and Output

Confusion Matrix

	ChatGPT	Grok	Copilot	DeepSeek
ChatGPT	185	3	88	20
Grok	1	272	8	15
Copilot	48	4	226	18
DeepSeek	5	12	21	258

RNN Input and Output

Confusion Matrix

	ChatGPT	Grok	Copilot	DeepSeek
ChatGPT	171	3	96	26
Grok	7	256	3	30
Copilot	97	6	177	16
DeepSeek	18	31	18	229

Now for the confusion matrices for both models when they received both the input and the output. We can see that most of the trends that we saw in the output

confusion matrices return as the models are best at predicting Grok and DeepSeek and still will mix up ChatGPT and Copilot.

Lessons & Experience

The biggest surprise for me was that I thought the CNN model would be worse than the RNN model because the CNN model could only look at these short word patterns of 3, 4, or 5 words while the RNN model was bidirectional and could see a lot more of the information at once. I thought because of the fact it had a lot more information to make its classification off it would be better at finding patterns in the long term and looking at sentence structure would make the RNN model better than the CNN model. Based on my results, however, it seems the better way to classify which LLM wrote the output would be to use the CNN model which was able to look at shorter patterns of word choice to be more accurate than the RNN model. Another thing that I found interesting is that the question made the RNN model slightly worse which was a bit surprising because as a human you think that seeing the question and response may help you predict because you'd think that the models may choose to answer similar questions in similar ways but that's not exactly how the models think about it and the models are instead trying to predict using the question data, which is just repeated across all 4 LLMs, making it useless noise that just ended up making the model worse. The CNN model was slightly better with the questions, but I attribute that to just randomness and think over repeated testing the difference between the CNN model with just the answer would be very slightly better than the CNN model with both the question and the answer. Lastly, I think it was very important that my models were unable to predict the LLM based on just the input as that would've undermined the point of this project of classifying which LLM wrote the output. In future projects, it is important to try and standardize variables like this so like how the models saw the same questions for all 4 LLMs any future project you would have something similar where a variable would have to be standardized across all the potential classifications so that variable doesn't have an impact on your model.